

Case Study Report:

Breast Cancer Classification

Table of Contents

1	Introduction	3
2	Data Collection and Pre-processing	4
2.1	Dataset Overview	4
2.2	Data Cleaning	5
2.3	Pre-Processing Steps.....	8
3	Design of Experiments	10
4	Descriptive Analytics and Information Visualization	12
5	Predictive Analytics Methods and Models.....	15
6	Testing, Validation, and Evaluation Metrics	18
7	Discussion of Accuracy and Efficiency	21
8	Conclusion of Solution	21
9	Appendices and References	22
9.1	Appendix A	22
9.2	References.....	23

1 Introduction

Breast cancer is a life-changing disease that occurs when abnormal cells in the milk ducts or lobules grow which can be unexpected and unpredictable forming tumours. If not treated in the right way, these tumours can spread to other parts of the body becoming life-threatening. It is the most common cancer affecting women, making up nearly 30% of all female cancer cases, according to Cancer Research UK¹ (Anon n.d.). Early detection is vital for improving outcomes and saving lives.

Mammographic screening, a widely used method involving X-ray imaging of breast tissue helps identify changes at an early stage. However, it is not always accurate, and some cases may go undetected. For women at higher risk, genetic testing and additional screening options are often recommended to improve the chances of early diagnosis (Cancer Support UK n.d.)

To address these challenges, the WHO Global Breast Cancer Initiative (GBCI) highlights the importance of promoting early detection, timely diagnosis and comprehensive treatment (World Health Organization 2024).

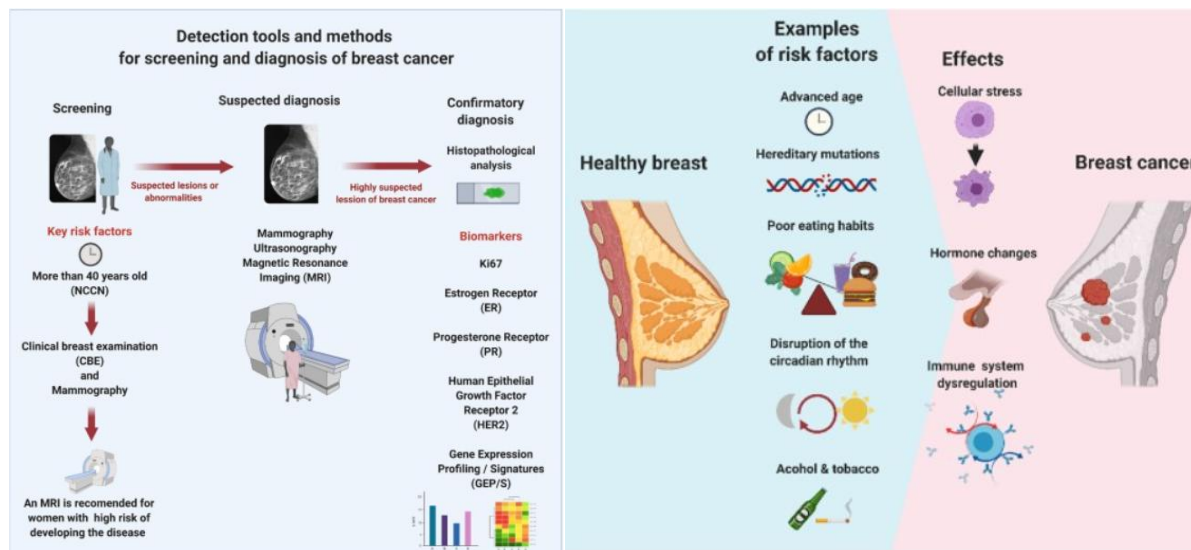


Figure 1 and 2 (Barba et al. 2021)

In this report, I've used Exploratory Data Analysis (EDA) and predictive analytics to analyse the Breast Cancer Wisconsin (Diagnostic) Data Set. The aim of this report is to determine whether a tumour in the breast mass is malignant or benign by leveraging machine learning algorithms to identify patterns and make precise predictions.

The key objectives would be to:

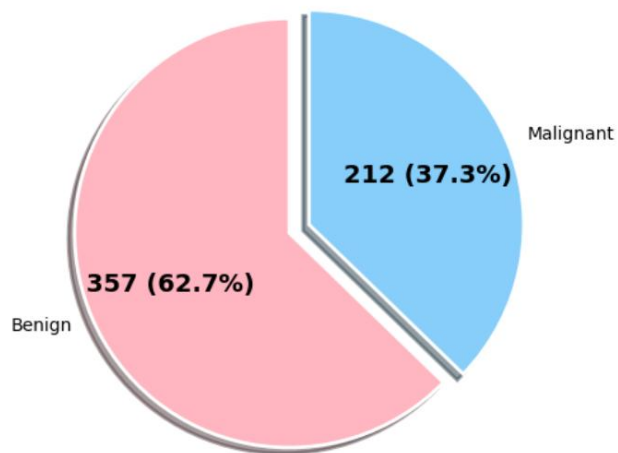
1. Conduct Exploratory Data Analysis, including data cleaning, handling outliers, analyzing feature correlations, and visualizing the dataset.
2. Analyze key features (radius, texture, perimeter, area) to classify tumours into two classes.
3. Develop and evaluate classification models to distinguish malignant (1) from benign (0) tumours.
4. Validate models using techniques like cross-validation, confusion matrix, and performance metrics (accuracy, precision, recall, Auc-Roc , F1-score) to ensure reliability and effectiveness.

2 Data Collection and Pre-processing

2.1 Dataset Overview

I've chosen the Breast Cancer Wisconsin (Diagnostic) Dataset to work on my problem of breast cancer classification. This dataset is ideal for binary classification perfectly aligning with my goal of classifying samples as malignant (class 1) or benign (class 0). It consists of 569 samples, each characterized by 30 features that describe the properties of cell nuclei extracted from breast mass images. The dataset is slightly imbalanced having 212 malignant samples (class 1) and 357 benign samples (class 0).

Distribution of Benign and Malignant Tumors



The dataset includes features that describe the size, shape, and structure of cells giving an idea of how they would look like. These features include the radius (mean distance from the centre to the cell boundary), texture, perimeter, area, smoothness, compactness (density and regularity of the cell shape), concavity (severity of concave portions on the boundary), the number of concave points, symmetry, and fractal dimension (complexity of the boundary). Each of these features is recorded in three ways: *the*

mean which represents the average value of the feature and gives an overall idea of the tumour's behaviour; *the standard error* which shows the variability or spread of the feature values—for example, a high standard error in smoothness or symmetry might indicate abnormal or cancerous growth *and the worst case* which shows the largest value of the feature detected in the image such as the cell with the most extreme radius or deepest concavity helping to identify outliers or indicate potential malignancy.

	radius1	texture1	perimeter1	area1	smoothness1	compactness1		radius2	texture2	perimeter2	area2	smoothness2	compactness2
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0	1.0950	0.9053	8.589	153.40	0.006399	0.04904
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	1	0.5435	0.7339	3.398	74.08	0.005225	0.01308
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	2	0.7456	0.7869	4.585	94.03	0.006150	0.04006
3	11.42	20.38	77.58	386.1	0.14250	0.28390	3	0.4956	1.1560	3.445	27.23	0.009110	0.07458
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	4	0.7572	0.7813	5.438	94.44	0.011490	0.02461

	concavity1	concave_points1	symmetry1	fractal_dimension1	...		concavity2	concave_points2	symmetry2	fractal_dimension2	radius2
0	0.3001	0.14710	0.2419	0.07871	...	0	0.05373	0.01587	0.03003	0.006193	25.38
1	0.0869	0.07017	0.1812	0.05667	...	1	0.01860	0.01340	0.01389	0.003532	24.99
2	0.1974	0.12790	0.2069	0.05999	...	2	0.03832	0.02058	0.02250	0.004571	23.57
3	0.2414	0.10520	0.2597	0.09744	...	3	0.05661	0.01867	0.05963	0.009208	14.91
4	0.1980	0.10430	0.1809	0.05883	...	4	0.05688	0.01885	0.01756	0.005115	22.54

Figure 3 Mean

Figure 4 Standard Error

	radius3	texture3	perimeter3	area3	smoothness3	compactness3			
0	25.38	17.33	184.60	2019.0	0.1622	0.11890			
1	24.99	23.41	158.80	1956.0	0.1238	0.08902			
2	23.57	25.53	152.50	1709.0	0.1444	0.08758			
3	14.91	26.50	98.87	567.7	0.2098	0.17300			
4	22.54	16.67	152.20	1575.0	0.1374	0.07678			

	concavity3	concave_points3	symmetry3	fractal_dimension3		
0	0.7119	0.2654	0.4601	0.11890		
1	0.2416	0.1860	0.2750	0.08902		
2	0.4504	0.2430	0.3613	0.08758		
3	0.6869	0.2575	0.6638	0.17300		
4	0.4000	0.1625	0.2364	0.07678		

Diagnosis	
0	M
1	M
2	M
3	M
4	M

Figure 5 Worse Case

Figure 6 Diagnosis (Target)

2.2 Data Cleaning

The data cleaning process started with checking for missing values in features and target, and none were found, which made things easier and allowed me to focus on preparing the data for modelling.

```

1 # Check for missing values, there is no missing value here which makes it easy to work on.
2 print("Missing values in features:\n", X.isnull().sum())
3 print("\nMissing values in target:\n", y.isnull().sum())
4 print("\nSummary Statistics:\n", X.describe())
5

```

Missing values in features:

radius1	0
texture1	0
perimeter1	0
area1	0
smoothness1	0
compactness1	0
concavity1	0
concave_points1	0
symmetry1	0
fractal_dimension1	0
radius2	0
texture2	0
perimeter2	0
area2	0
smoothness2	0
compactness2	0
concavity2	0
concave_points2	0

Figure 7 No Missing Values Found

I've also used the describe function to get a comprehensive overview of the dataset, helping to identify outliers, the need of feature scaling because of large difference of scaling between features such as the large range of radius1 compared to the small range of smoothness1 to ensure that no single feature dominates the learning process in the model, as well as count which confirms that there are no missing values.

Summary Statistics:					
	radius1	texture1	perimeter1	area1	smoothness1 \
count	569.000000	569.000000	569.000000	569.000000	569.000000
mean	14.127292	19.289649	91.969033	654.889104	0.096360
std	3.524049	4.301036	24.298981	351.914129	0.014064
min	6.981000	9.710000	43.790000	143.500000	0.052630
25%	11.700000	16.170000	75.170000	420.300000	0.086370
50%	13.370000	18.840000	86.240000	551.100000	0.095870
75%	15.780000	21.800000	104.100000	782.700000	0.105300
max	28.110000	39.280000	188.500000	2501.000000	0.163400

A key part of cleaning the data was examining outliers, here I've shown the outliers of the mean values of features like radius1 and area1 using boxplots which also showed an overview of how each feature is distributed. At first, outliers seemed like they might just be noise, but I realized they could actually provide critical insights—especially in rare cases. For example, a tumour with a significantly large radius or area might indicate a rare and aggressive malignant tumour. Rather than removing these extreme points, I decided to keep them in the dataset as removing them might have simplified the data but would have risked reducing the model's ability to generalize and correctly classify cases. Including these outliers ensured the model could learn to identify such rare cases which are vital for distinguishing malignant tumors from benign ones.

```

Looking for Outliers

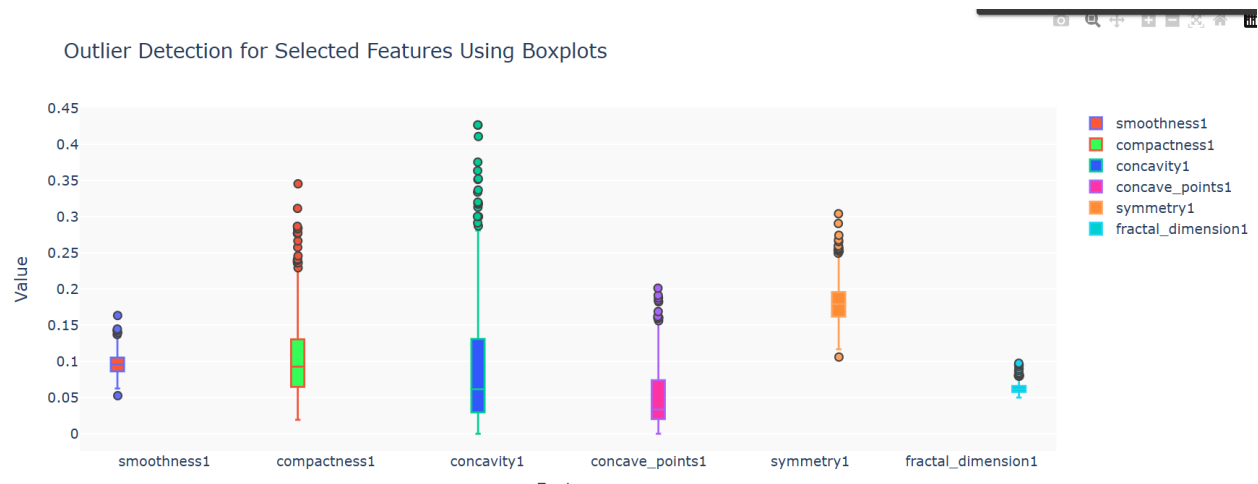
1 # Select the important features excluding area1, radius1, perimeter1, texture1
2 selected_features = ['radius1', 'area1', 'smoothness1', 'compactness1', 'concavity1', 'concave_points1', 'symmetry1', 'fractal_dimension1']
3
4 # Convert the selected features to a list of lists for plotting
5 values = [X[feature].tolist() for feature in selected_features]
6
7 # Define vibrant colors for each feature
8 vibrant_colors = ['#FF5733', '#33FF57', '#3357FF', '#FF33A1', '#FF8C33', '#00CED1']
9
10 # Create the Plotly figure
11 fig = go.Figure()
12
13 # Loop through the features, adding a box trace for each one
14 for feature_name, feature_values, color in zip(selected_features, values, vibrant_colors):
15     fig.add_trace(go.Box(
16         y=feature_values,
17         name=feature_name,
18         boxpoints='outliers', # Show outliers
19         jitter=0.1,          # Increase jitter to spread out points more evenly
20         whiskerwidth=0.4,
21         fillcolor=color,
22         marker=dict(size=8, line=dict(width=1.5)),
23         line=dict(width=2)
24     ))
25
26 # Set figure layout for better aesthetics
27 fig.update_layout(
28     title='Outlier Detection for Selected Features Using Boxplots',

```

Figure 8 Outliers (snippet from notebook)



Figure 9 Outliers using Boxplot for mean radius and mean area



2.3 Pre-Processing Steps

```

Checking what are the values in the Diagnosis column to perform label encoding to change categorical values to numerical

[8] 1 # Check the column name(s) in targets
    2 print(breast_cancer_wisconsin_diagnostic.data.targets.columns)
    3
    4 # Assuming the column is named 'Diagnosis', get unique values
    5 print("Unique values in 'Diagnosis':", breast_cancer_wisconsin_diagnostic.data.targets['Diagnosis'].unique())

Index(['Diagnosis'], dtype='object')
Unique values in 'Diagnosis': ['M' 'B']

```

To prepare the dataset for machine learning, I used label encoding to convert the categorical target variable Diagnosis, into numerical values to ensure the dataset is compatible with the machine learning

algorithms used later for predictive analysis. The diagnosis column originally contained two categories: M for malignant and B for benign. After label encoding, these categories were changed into 1 for malignant and 0 for benign as shown below:

▼ Changing M → 1 (cancerous) and B → 0 (non-cancerous)

```
1 # Use .loc to modify the Diagnosis column
2 breast_cancer_wisconsin_diagnostic.data.targets.loc[:, 'Diagnosis'] = breast_cancer_wisconsin_diagnostic.data.targets['Diagnosis'].map({'M': 1, 'B': 0})
3
4 # Verify the conversion by checking unique values
5 print("Unique values in 'Diagnosis' after encoding:", breast_cancer_wisconsin_diagnostic.data.targets['Diagnosis'].unique())
```

Unique values in 'Diagnosis' after encoding: [1 0]

▼ Checking if label encoding has been applied successfully to proceed with further steps - *SUCCESSFUL*

```
1 # Display the first 15 rows of the target variable (y) - checking if label encoding was successful.
2
3 print(y.head(20))
```

Diagnosis

0	1
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	1
13	1
14	1
15	1
16	1
17	1
18	1
19	1

After encoding the target variable, I split the dataset into 80% training and 20% testing sets to help the model learn effectively while keeping a portion aside to test its performance. Since some features, like `area1` had much larger ranges than others, I applied a standard scaler to both the sets which centres them around the mean and scales them to unit variance. This scaling process made sure that all features were treated equally, preventing any one feature from dominating the others due to its size or range. By standardising the data, the model could focus on identifying meaningful patterns rather than being influenced by features with extreme values improving its performance.

▼ Splitting dataset into 80% training and 20% testing sets

```
1 # Split the data into training and testing sets
2 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
3 X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

((455, 30), (114, 30), (455, 1), (114, 1))

▼ Using StandardScaler to ensure that all features contribute equally which improves the model's performance

```
1 from sklearn.preprocessing import StandardScaler
2
3 # Standardize the features using StandardScaler
4 scaler = StandardScaler()
5
6 # Fit the scaler on the training data and transform both training and testing data
7 X_train = scaler.fit_transform(X_train) # Fit and transform the training set
8 X_test = scaler.transform(X_test)
```

3 Design of Experiments

Data Science **OSEMN** Model

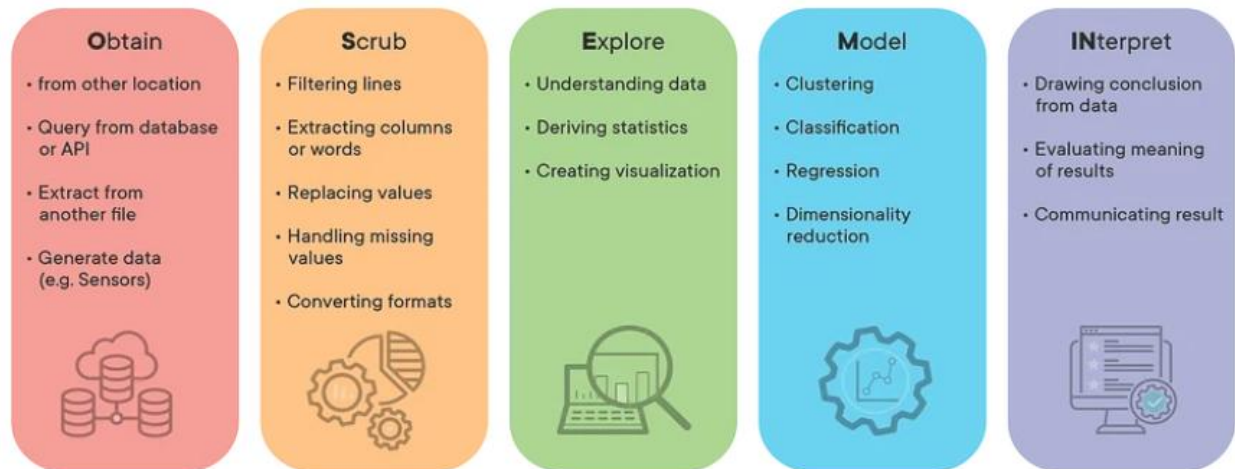


Figure 10 (Ramirez 2021)

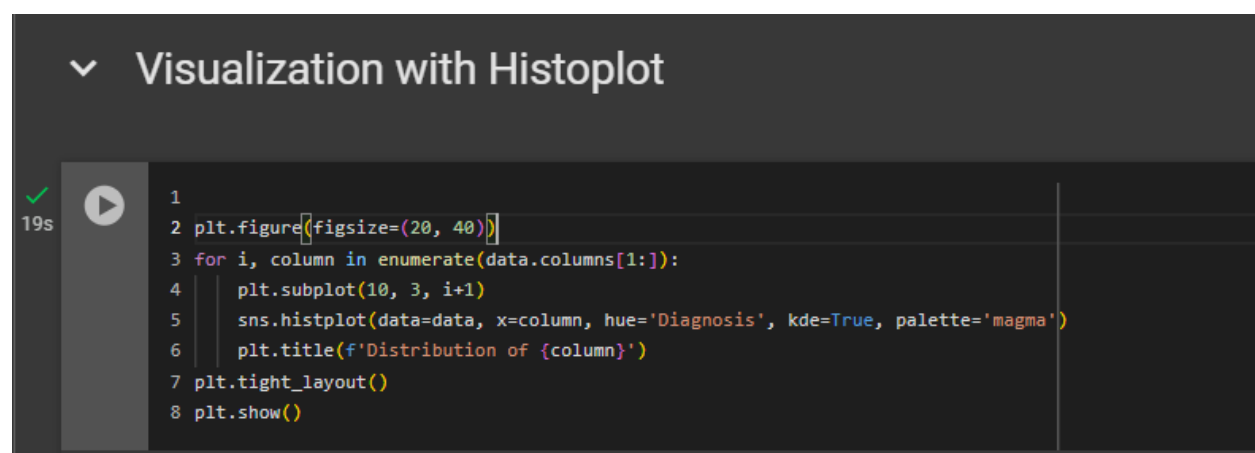
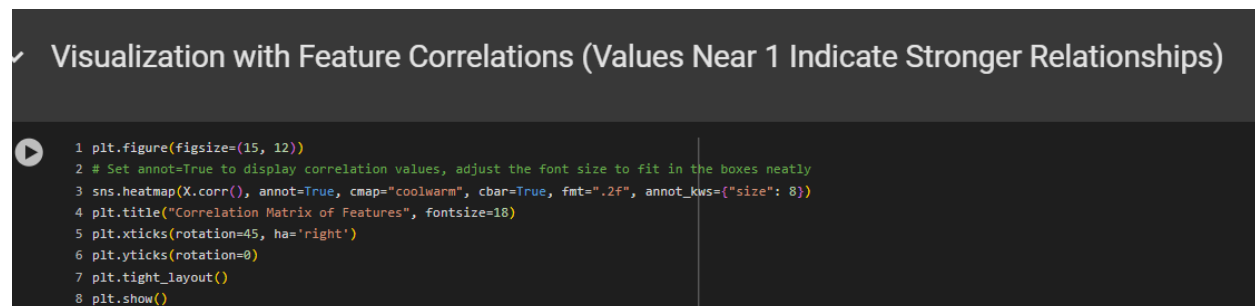
I decided to use the OSEMN methodology for this assignment because it's simple, easy to follow, and fits well with the workflow of data science projects. What I like most about it is its flexibility, you can always go back to a previous step if something doesn't work out, which is really helpful when dealing with the trial-and-error nature of machine learning models. The methodology breaks the process into five clear steps to stay organized and focused:

Obtain: This step is all about gathering the dataset and getting familiar with it. I used the Breast Cancer Wisconsin (Diagnostic) Dataset which I imported from the UC Irvine Machine Learning Repository—a popular platform in the machine learning community for sharing and donating datasets. I spent time understanding the dataset's structure, including its features and target variable to make sure I could make informed choices that would improve the model's accuracy.

```
▼ Importing the Dataset - radius1 -> mean, radius2 -  
  
1 from ucimlrepo import fetch_ucirepo  
2 # fetch dataset  
3 breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)  
4  
5 # data (as pandas dataframes)  
6 X = breast_cancer_wisconsin_diagnostic.data.features  
7 y = breast_cancer_wisconsin_diagnostic.data.targets  
8  
9 # metadata  
10 print(breast_cancer_wisconsin_diagnostic.metadata)  
11  
12 # variable information  
13 print(breast_cancer_wisconsin_diagnostic.variables)
```

Scrub: This step involves cleaning and preparing the dataset to ensure it's ready for modelling like mentioned in 3 Data Cleaning and Pre-processing.

Explore: This step consists of using visualizations, I used libraries like Matplotlib, Seaborn, and Plotly to create visualizations such as heatmaps for feature correlations, pie charts for categorical distributions, histograms for feature distributions, barplots for outliers and pairplots to analyze relationships between the features.



Model: I chose a Feedforward Neural Network and Support Vector Classifier with Grid Search and Cross-Validation. According to my research and experiment with different models such as KNN mentioned in Appendix A, these models had good accuracies. These can handle complex patterns in the

data, manage class imbalances by adjusting weights and thresholds and perform well in binary classification problems. Using multiple models allowed for a thorough evaluation of the dataset and ensured the selection of the most effective approach for accurately classifying breast tumour into class 1 or 0 after analysing features of cell nuclei in the breast mass.

Interpret: In this step, I evaluated the models to compare their performance and determine the best one for my aim. Performance Metrics like confusion matrix for training and testing sets, precision, recall, F1 score, and ROC curve were used, recall was particularly important for this problem as correctly identifying malignant tumours is critical to avoid false negatives.

4 Descriptive Analytics and Information Visualization

Feature Correlation with Heatmap:

The heatmap shows the relationships between different features in the dataset using colours to show how closely related they are. Darker blue boxes show strong correlations which are close to 1 while lighter colours show weak correlations. For example, radius1, perimeter1, area1, radius3, perimeter3 and area3 are strongly related having a value of 95 and above meaning that when one feature's value rises, the others are likely to rise as well, this also reflects similar size or geometric properties of the samples. Similarly, compactness1, concavity1, and concave_points1 are also closely linked showing how compact or irregular the samples are. On the other hand, features like symmetry1, fractal_dimension1 and fractal_dimension3 show lower correlations with other features meaning they provide unique information that might not directly depend on size or shape and can identify other important patterns such as the symmetry or complexity of the sample's structure which could be critical for classification.

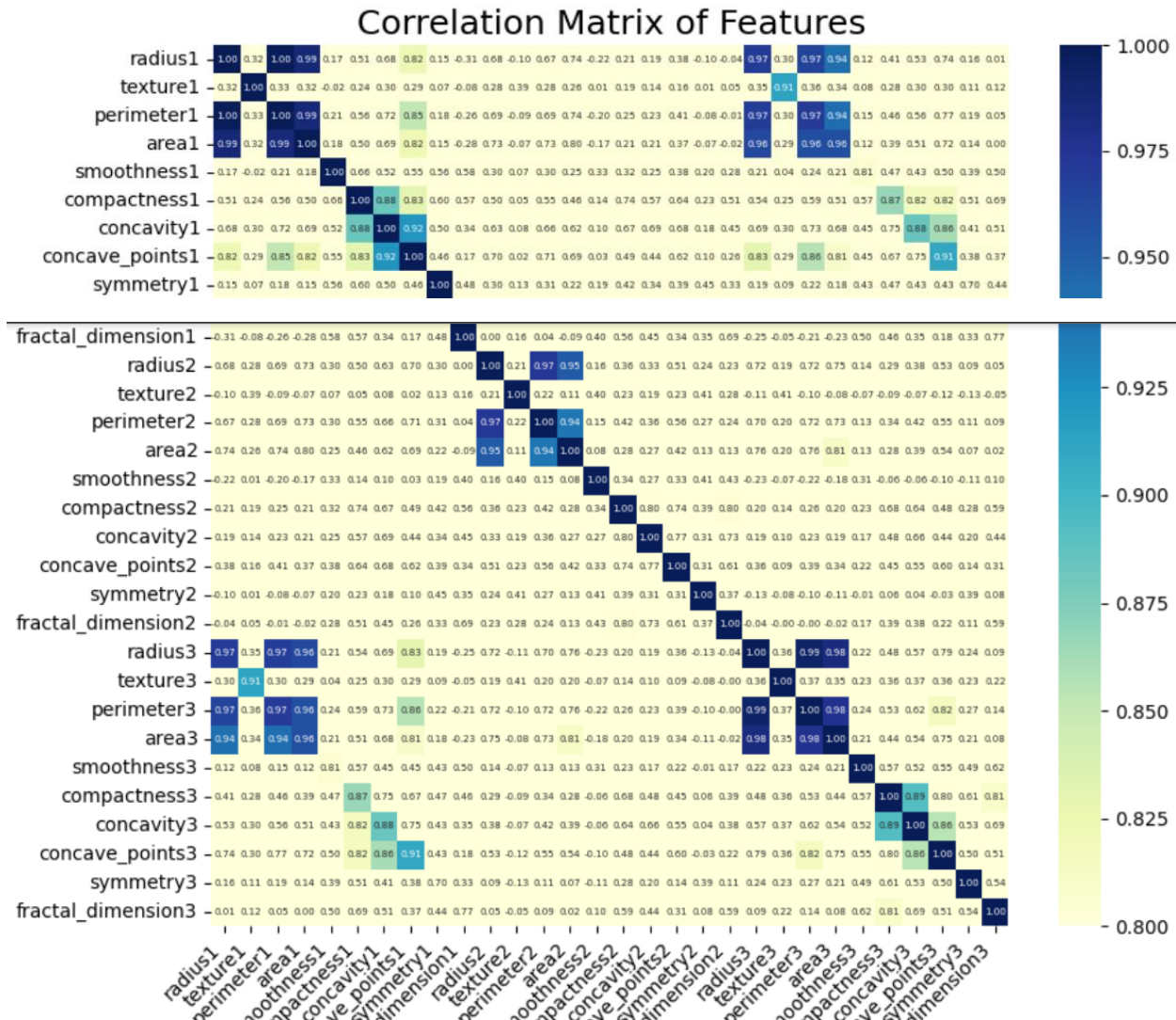


Figure 11 Visualizations with Heatmap (Feature Correlations) Highlights correlation 0.80 +

Pair plots:

The pair plot gives a clear view of how different features relate to each other and how they are distributed for benign and malignant cases. Features like radius and perimeter have a strong linear relationship where the points are closely aligned in an almost straight line showing that these features are highly related, this also shows that malignant samples tend to have larger values for radius and perimeter whereas benign samples are comparatively smaller and smoother. On the other hand, some relationships such as between concave_points and radius or perimeter are more scattered and less linear. This scattering shows that while these features are related, their relationship is not simple or linear. Instead, they show more complex patterns in the data such as differences in the tumour's shape, structure or texture.

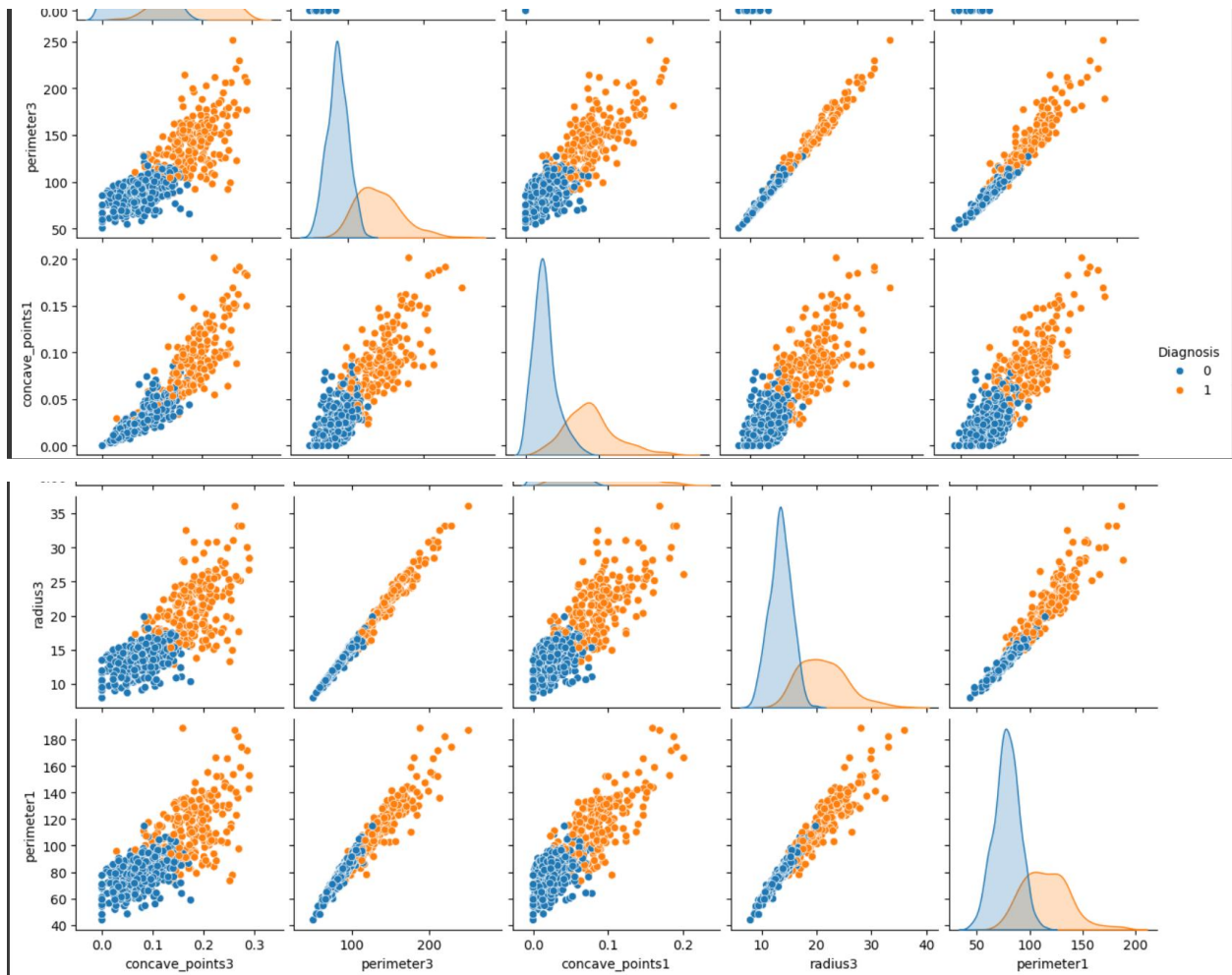


Figure 12 Visualizations with Pair Plots

Histplots:

The histograms show the differences between benign and malignant tumours by showing how key features vary across the two classes. Features like perimeter, area, and concave_points show clear separation with malignant tumours having higher values. This reflects their larger size and more irregular structure making these features strong indicators for malignant cases.

The smoothness and compactness show more overlap between the two classes making them less important for differentiating both classes but still useful for providing additional insights. Similarly, concavity and concave_points show that malignant tumors often have higher values, further supporting their irregularity. For texture-related features such as texture1, there's moderate overlap but malignant tumours tend to have slightly higher values indicating differences in the tumour's surface.

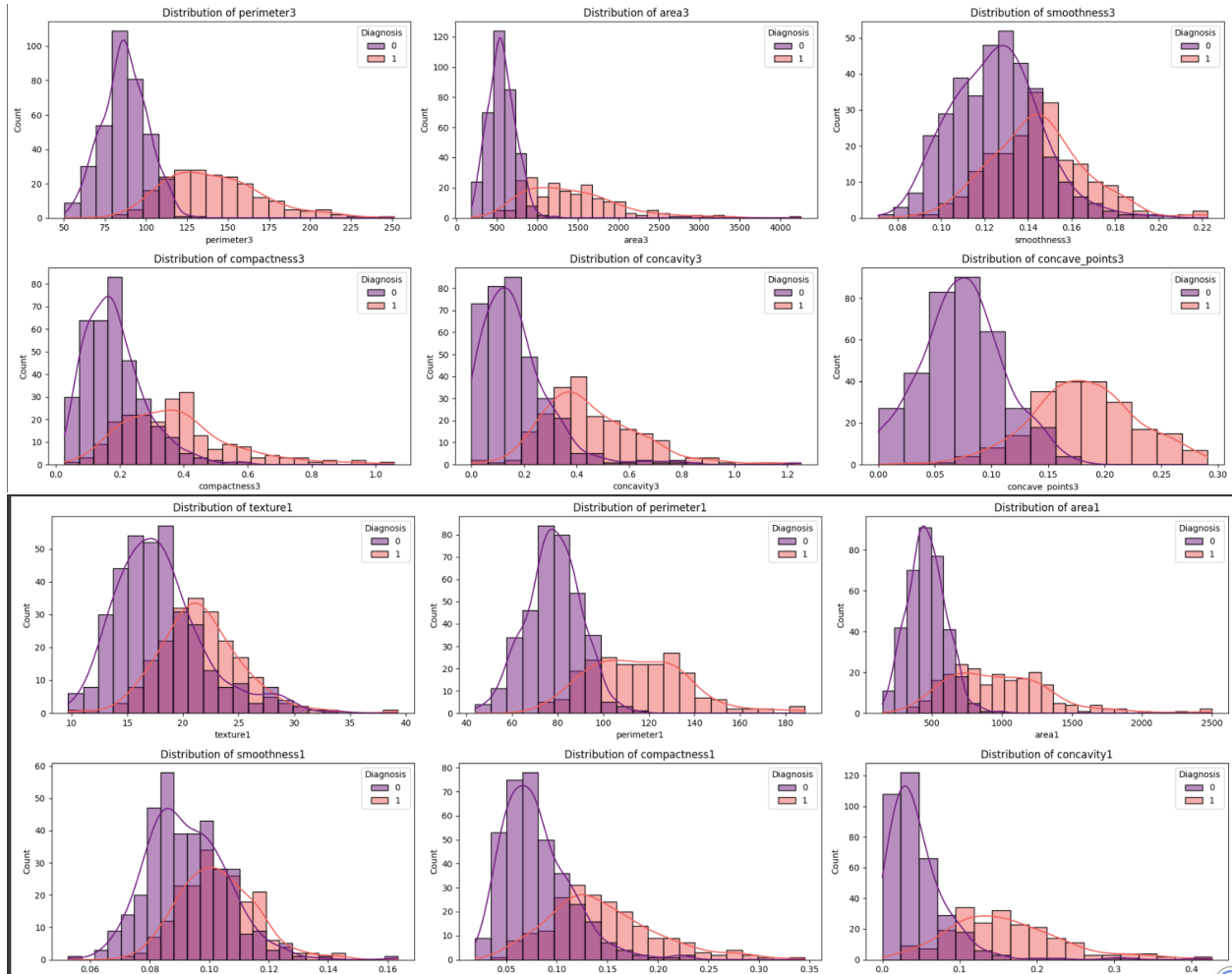


Figure 13 Visualization with Histplots

5 Predictive Analytics Methods and Models

Feedforward Neural Network (Multi-Layer Perceptron - MLP):

This artificial neural network processes information in one direction from the input layer to the output layer which could include hidden layers and has no feedback loops. I chose this model because it can effectively capture and model complex, non-linear relationships within the data. For instance, the relationship between a cell nuclei's smoothness and texture may be non-linear, as nuclei with rougher textures don't always have a proportional increase in smoothness due to abnormal growth.

I used Grid Search with cross-validation to find the optimal combination of hyperparameters for the model which resulted in an accuracy of 97.36%. Grid Search tested different combinations of parameters such as the hidden layer sizes, activation functions, solvers, learning rates, and maximum iterations as shown in the parameter grid. Cross-validation was performed using the 5-fold method, meaning the data

was split into 5 sets, the model is trained on the 4 sets and tested on the 5th one. This was repeated for all 5 sets to ensure the results were reliable and not just biased towards one set.

Hidden Layer Sizes (100, 50): The model performed better with the 1st hidden layer with 100 neurons and the 2nd with 50. This multi-layer structure helped identify complex patterns within the features, while the smaller second layer helped minimise overfitting by simplifying the data representation as it progresses through the network.

Activation Function: ReLU was chosen as it delivered better performance by reducing false negatives, effectively handling the non-linear relationships in the data, and avoiding the vanishing gradient problem. Although tanh performed slightly better on the testing set, it had 7 false negatives for Class 1 (malignant) in the training set, which is a critical limitation in medical diagnostics where accurately identifying malignant cases is essential.

Learning Rate and Maximum Iterations: The learning rate which shows the step size for adjusting weights during training was set to 0.01 for faster convergence without exceeding optimal values. Maximum iterations/epochs which is the number of training cycles the model completes were set to 1000 to ensure sufficient training.

```

  ✓ Feedforward Neural Network (Multi-Layer Perceptron)

1 from sklearn.neural_network import MLPClassifier
2 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, roc_auc_score
3 from sklearn.model_selection import GridSearchCV
4
5 # Define the parameter grid for Grid Search
6 param_grid = {
7     'hidden_layer_sizes': [(50, ), (100, ), (100, 50)],
8     'activation': ['relu', 'tanh'],
9     'solver': ['adam', 'lbfgs'],
10    'learning_rate_init': [0.0001, 0.001, 0.01, 0.04, 1],
11    'max_iter': [1000, 2000, 5000],
12    'early_stopping': [True], #stops training if it does not improve for a set number of epochs preventing overfitting and unnecessary training epochs
13 }
14
15 # Initialize the MLP classifier
16 mlp_base = MLPClassifier(random_state=42)
17
18 # Set up Grid Search with cross-validation
19 grid_search = GridSearchCV(estimator=mlp_base, param_grid=param_grid, cv=5, scoring='accuracy', verbose=1)
20 grid_search.fit(X_train, y_train.values.ravel())
21
22 # Best parameters from Grid Search
23 best_params = grid_search.best_params_
24 print("Best parameters from Grid Search:", best_params)
25

Fitting 5 folds for each of 180 candidates, totalling 900 fits
Best parameters from Grid Search: {'activation': 'relu', 'early_stopping': True, 'hidden_layer_sizes': (100, 50), 'learning_rate_init': 0.01, 'max_iter': 1000,
```

Support Vector Classifier:

After the Grid Search and cross-validation with the same method above, these were the best hyperparameters chosen for SVM achieving the same accuracy on testing set as the FNN with slight differences in performance metrics. Unlike FNN, which requires tuning multiple parameters like hidden layers and activation functions, SVM focuses on fewer parameters like the kernel and regularization. SVM is also simpler, more traditional and easier to interpret, making it a practical alternative for binary classification problems especially with smaller datasets like this one.

A few of those parameters are:

Regularization Parameter: The grid search tested different values of C from 0.01 to 20 to find the best balance between model simplicity and performance. 0.01 was chosen because it creates a simpler decision boundary allowing for a few misclassifications in the training set but helps the model avoid overfitting and perform better on testing set. Larger values like 10 or 20 reduces bias because the model fits well to the training data and it increases variance leading to low accuracy on testing data.

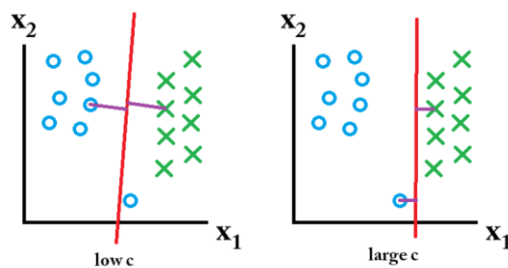


Figure 14 (Anon 2012)

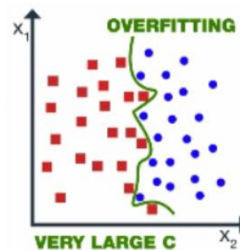


Figure 15 (S 2021)

Kernels and Gamma: The grid search tested different kernel options (linear, rbf, poly, sigmoid) and gamma values (scale, auto, 0.0001 to 0.1). The linear kernel was chosen because it works well for linearly separable data and suits smaller datasets like this. For gamma, scale was selected as it adjusts automatically based on the number of features providing a good balance without the risk of overfitting or underfitting unlike fixed gamma values that need careful tuning.

Class Weight: To address the slight class imbalance in the dataset, The `class_weight='balanced'` option was selected as it automatically adjusts the importance of each class inversely proportional to their frequencies to make sure equal importance was given to the minority class (malignant).

```

1 from sklearn.svm import SVC
2 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_auc_score, roc_curve, precision_recall_curve
3
4 # Define the parameter grid for Grid Search
5 param_grid = {
6     'C': [0.01, 0.05, 0.5, 0.1, 1, 10, 15, 20],
7     'kernel': ['linear', 'rbf', 'poly', 'sigmoid'], #rbf, poly for non linearly
8     'gamma': ['scale', 'auto', 0.0001, 0.001, 0.01, 0.1],
9     'class_weight': [None, 'balanced', {0: 1, 1: 5}] #Adjusts the importance of classes to handle class imbalance:'balanced' (auto adjust weights based on class frequencies
10 } # or custom weights ({0: 1, 1: 5}) to give higher importance to the minority class eg malignant cases
11
12 # Perform Grid Search with Cross Validation
13 grid_search = GridSearchCV(estimator=SVC(probability=True), param_grid=param_grid, cv=5, scoring='accuracy')
14 grid_search.fit(X_train, y_train.values.ravel())
15
16 # Get the best hyperparameters
17 best_params = grid_search.best_params_
18 print("Best Hyperparameters from Grid Search:", best_params)
19
20 # Initialize and train the SVM model with the best parameters
21 svm_model = SVC(**best_params, probability=True)
22 svm_model.fit(X_train, y_train.values.ravel())
23
24 # Predict probabilities for the positive class (Malignant)
25 #The model predicts probabilities for both classes, and we extract the probabilities for the malignant class (positive class).
26 y_pred_proba = svm_model.predict_proba(X_test)[:, 1]
27
28 #Decision Threshold
29 #Sets a threshold value (0.5 by default) to classify the predictions.
30 #If the predicted probability for the malignant class is greater than or equal to the threshold, classify it as malignant (1) Otherwise, classify it as benign (0).
31 threshold = 0.5
32 y_pred_threshold = (y_pred_proba >= threshold).astype(int)
33

```

```

Best Hyperparameters from Grid Search: {'C': 0.01, 'class_weight': 'balanced', 'gamma': 'scale', 'kernel': 'linear'}

```

6 Testing, Validation, and Evaluation Metrics

```

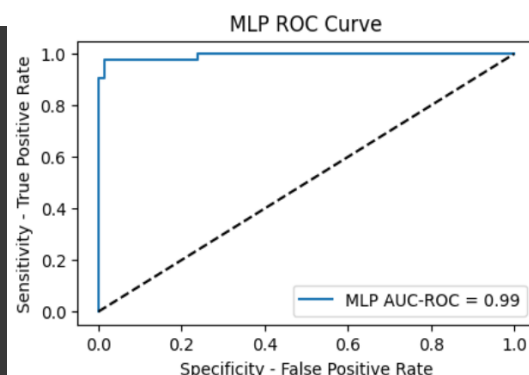
MLP Accuracy: 0.9736842105263158
      precision    recall  f1-score   support

      0       0.99      0.97      0.98        71
      1       0.95      0.98      0.97        43

   accuracy          0.97          114
  macro avg       0.97      0.97      0.97        114
 weighted avg       0.97      0.97      0.97        114

MLP Training Accuracy: 0.9956043956043956
MLP Testing Accuracy: 0.9736842105263158

```

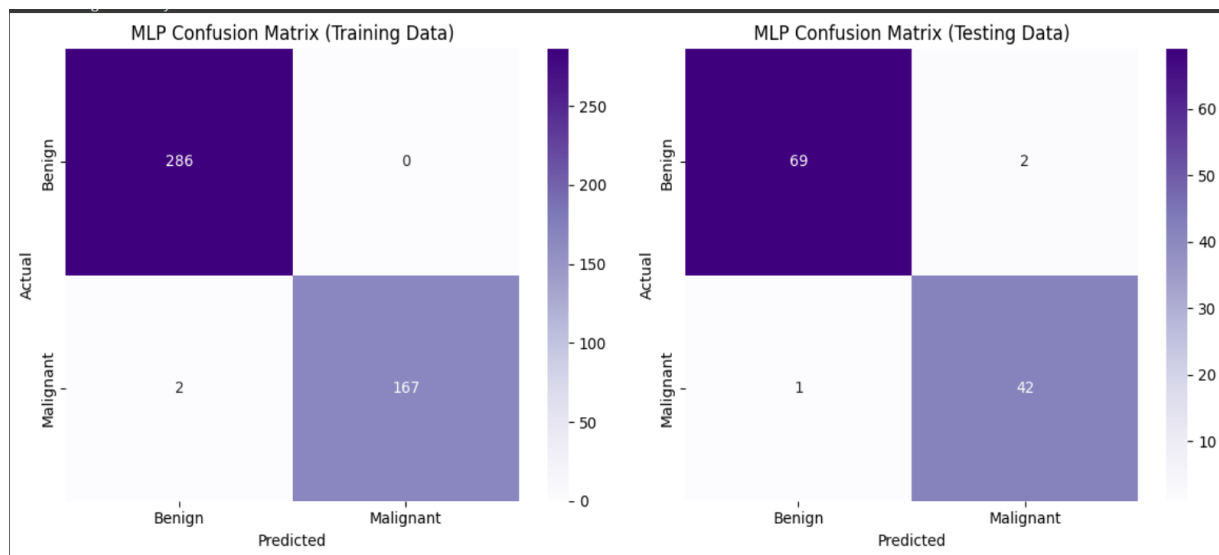


The MLP model performed quite well in both training and testing and achieved a high training accuracy of 99.56% showing that it identified patterns in the dataset without overfitting, this was achieved by focusing on minimising the cross-entropy loss (log loss function used mostly in binary classification) during training which is how closely the model's predicted probabilities match the true labels (Anon 2024a). On the testing set, it maintained a strong accuracy of 97.37% showing that it works well with unseen data. The ROC curve further backed this up, with an AUC-ROC score of 0.99 which shows its excellent ability to distinguish between benign and malignant cases.

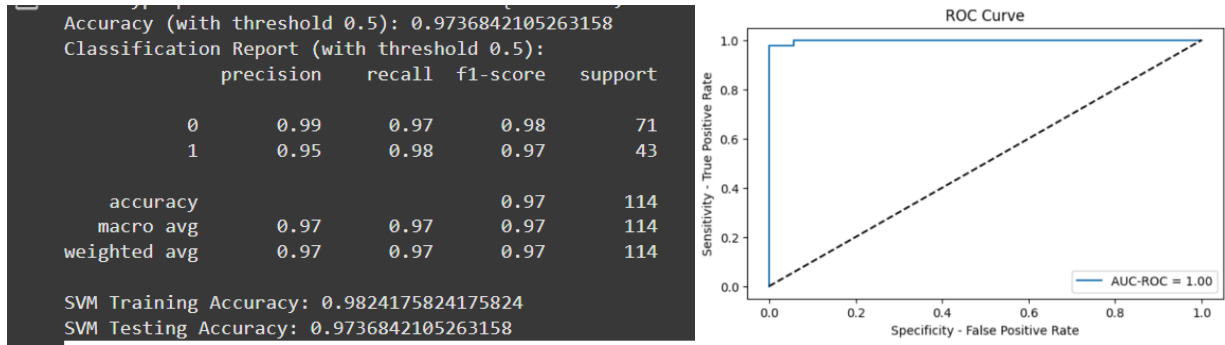
Metric	Definition	Use Case
Accuracy	The proportion of correctly classified instances (both true positive and true negative) over all instances.	Measures the overall performance of a classifier
Precision	The proportion of correctly classified positive instances over all instances that are classified as positive.	Measures the ability of the classifier to avoid false positives
Recall	The proportion of correctly classified positive instances over all actual positive instances.	Measures the ability of the classifier to identify all actual positive instances
F1-Score	The harmonic mean of precision and recall, providing a balanced measure of both precision and recall.	A good indicator of the performance of a classifier when the number of positive and negative instances is unbalanced

Figure 15 (Singh 2023)

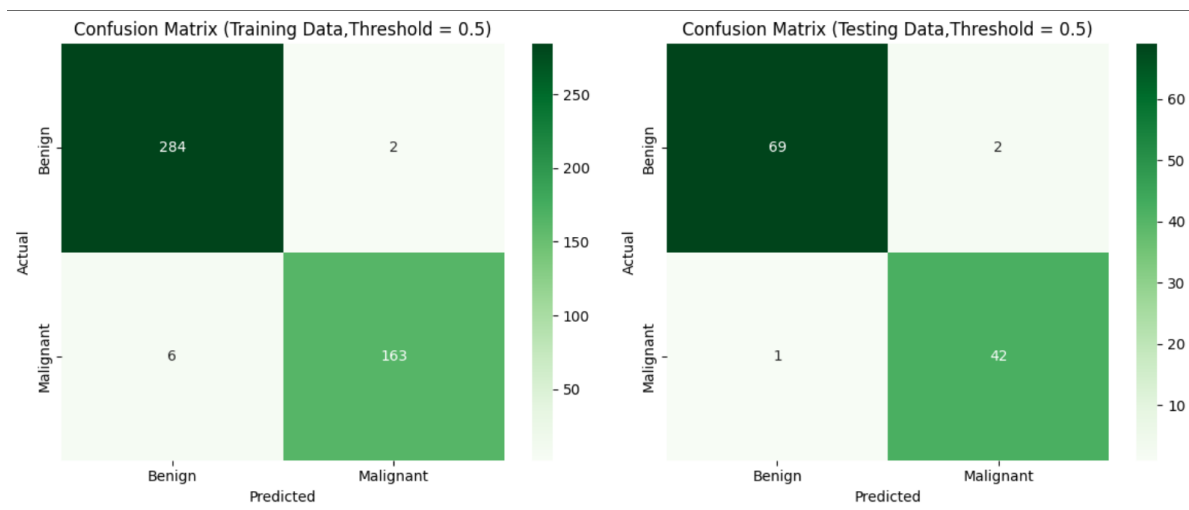
The model showed better precision for class 0 (Benign) which means it predicts most of the positive benign cases accurately. It had a higher recall for class 1 (Malignant) showing strong ability to correctly identify malignant cases from positive which is crucial in medical diagnoses. The F1-scores were very close, with 0.98 for benign and 0.97 for malignant showing how consistent and reliable the model is at handling both types of cases.



From the confusion matrix, the model made just 2 false negatives and no false positives during training. On the testing set, it misclassified only 1 malignant case as benign and 2 benign cases as malignant, which is still very accurate.



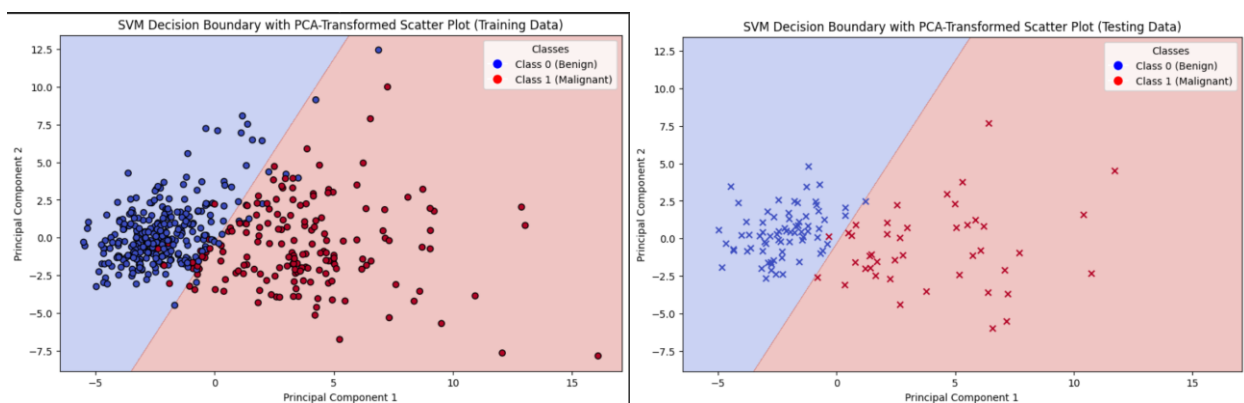
The SVM model performed well as well achieving a training accuracy of 98.24% slightly lower than the MLP which might be because it generalizes better by avoiding overfitting on training data giving a higher FN. On the testing set, it had the same accuracy as MLP of 97.37%. The SVM had better ROC score of 1.00 compared to MLP which shows it can be a better choice than MLP in terms of distinguishing between classes at various thresholds. It showed the same performance metrics as the MLP model for both class 0 and class 1 performing equally well in terms of precision, recall, and F1-score for both classes.



The confusion matrices showed that the SVM model had slightly more misclassifications than the MLP model during training, with 6 false negatives and 2 false positives compared to the MLP's matrix. This could be because of a lower c value focusing on maximizing the margin between classes which might have caused it to misclassify some edge cases in the training data. However, on the testing set both models had identical performance, each misclassifying 1 malignant case as benign and 2 benign cases as malignant.

7 Discussion of Accuracy and Efficiency

Both models performed well in terms of accuracy, but a key difference would be between balancing accuracy and computational efficiency. While the MLP achieves high accuracy, it requires more computational resources such as GPU power, time and RAM due to its iterative backpropagation process which is updating weights through multiple layers to minimise the error and the need for hyperparameter tuning. I would say SVM is more resource-efficient making it a suitable option for smaller datasets like this one but its performance is heavily dependent on selecting the right parameters such as the kernel type, regularization and gamma values.



Ive used decision boundary visualized with PCA-transformed data to understand how SVM separates benign (class 0) and malignant (class 1) cases.

In the training scatter plot, the model creates a clear linear boundary correctly classifying most points into their respective regions with only a few misclassifications near the boundary. This aligns with the false negatives and false positives seen in the training confusion matrix above.

In the testing scatter plot, the decision boundary remains consistent showing the SVM model generalizes well to new data. Most benign and malignant cases are correctly classified with only a small number of misclassified points close to the boundary supporting the testing accuracy of 97.37%

8 Conclusion of Solution

To improve accuracy, a few improvements could be; SMOTE (Synthetic Minority Oversampling Technique) can be applied to address the slight class imbalance instead of adjusting weights/threshold ensuring both SVM and MLP models avoid bias toward the majority class and improve their ability to correctly classify malignant cases (saikat 2021). For SVM incorporating feature selection techniques like recursive feature elimination (RFE) selecting highly correlated features could help reduce noise

(avcontentteam 2023) and for MLP, advanced hyperparameter optimization methods and regularization techniques like dropout could improve the model's stability and accuracy (Singh 2020).

9 Appendices and References

9.1 Appendix A

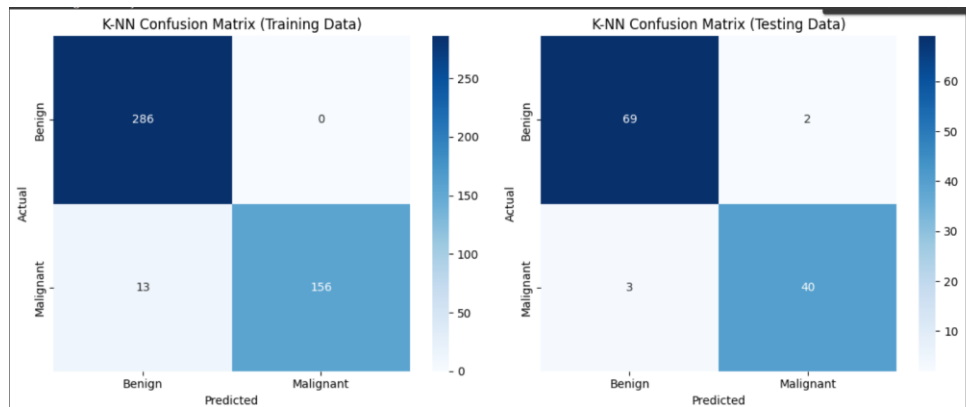
K-Nearest Neighbors (KNN):

```
1 from sklearn.neighbors import KNeighborsClassifier
2
3 # Initialize and train the K-NN model
4 knn_model = KNeighborsClassifier(n_neighbors=10)
5 knn_model.fit(X_train, y_train.values.ravel()) #ravel used to change from 2d to 1d array to ensure compatibility with the KNN model's fit
6
7 # Make predictions on the test set
8 y_pred_knn = knn_model.predict(X_test)
9
10 # Evaluate the K-NN model
11 accuracy_knn = accuracy_score(y_test, y_pred_knn)
12 print(f"K-NN Accuracy: {accuracy_knn}")
13
14 # Classification report for K-NN
15 print(classification_report(y_test, y_pred_knn))
16
17 # Train accuracy for KNN
18 knn_train_accuracy = knn_model.score(X_train, y_train)
19 print(f"KNN Training Accuracy: {knn_train_accuracy}")
20
21 # Test accuracy for KNN
22 print(f"KNN Testing Accuracy: {accuracy_knn}")
23
24
25 # Confusion Matrix for K-NN (Training Data)
26 y_pred_train_knn = knn_model.predict(X_train)
27 cm_train_knn = confusion_matrix(y_train, y_pred_train_knn)
```

K-NN Accuracy: 0.956140350877193

	precision	recall	f1-score	support
0	0.96	0.97	0.97	71
1	0.95	0.93	0.94	43
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

KNN Training Accuracy: 0.9714285714285714
KNN Testing Accuracy: 0.956140350877193



9.2 References

- Anon (2024a) *1.4. Support Vector Machines*. <https://scikit-learn.org/1.5/modules/svm.html#unbalanced-problems> Accessed 25 December 2024.
- Anon (2024b) *1.17. Neural network models (supervised)*. https://scikit-learn.org/1.5/modules/neural_networks_supervised.html Accessed.
- Anon (2024c) *Feedforward neural network*. <https://www.geeksforgeeks.org/feedforward-neural-network/> Accessed.
- Anon (2012) *machine learning - What is the influence of C in SVMs with linear kernel?* <https://stats.stackexchange.com/questions/31066/what-is-the-influence-of-c-in-svms-with-linear-kernel> Accessed.
- Anon (2024d) *StandardScaler*. <https://scikit-learn.org/1.5/modules/generated/sklearn.preprocessing.StandardScaler.html> Accessed.
- avcontentteam (2023) *Recursive Feature Elimination: Working, Advantages & Examples*. <https://www.analyticsvidhya.com/blog/2023/05/recursive-feature-elimination/> Accessed.
- Barba, D., León-Sosa, A., Lugo, P., Suquillo, D., Torres, F., Surre, F., Trojman, L. and Caicedo, A. (2021) Breast cancer, screening and diagnostic tools: All you need to know. *Critical Reviews in Oncology/Hematology* 157 (103174), 103174.
- Cancer Support UK (n.d.) *Breast cancer: understanding the symptoms, causes and treatment*. https://cancersupportuk.org/breast-cancer-understanding-the-symptoms-causes-and-treatment/?utm_source=google&utm_medium=cpc&utm_campaign=18712285383&utm_content=143267181256&utm_term=breast%20cancer&gad_source=1&gclid=Cj0KCQiAgdC6BhCgARIsAPWNWH0GORFTccV6oYBsW5jvcDM9tWTcg4fU2DWe-Y5lmOcF5tiUJETLq_EaAnlbEALw_wcB Accessed.

- chanchal24 (2024) *Breast Cancer Prediction with 7+ Models 98% ACC*.
Kaggle<https://www.kaggle.com/code/chanchal24/breast-cancer-prediction-with-7-models-98-acc/notebook#SVC> Accessed 25 December 2024.
- habibamowafy (2024) *Breast_Cancer SVM Model*.
Kaggle<https://www.kaggle.com/code/habibamowafy/breast-cancer-svm-model/notebook>
Accessed 8 November 2024.
- ironwolf437 (2024) *breast cancer wisconsin - ANN*.
Kaggle<https://www.kaggle.com/code/ironwolf437/breast-cancer-wisconsin-ann/notebook#ANN-Model> Accessed 25 December 2024.
- Karanam, S. (2022) *Exploratory Data Analysis — Breast Cancer Wisconsin (Diagnostic) Dataset*.
<https://medium.com/@shashmikiranam/exploratory-data-analysis-breast-cancer-wisconsin-diagnostic-dataset-6a3be9525cd> Accessed.
- msripooja (2019) *Breast cancer prediction using SVM*.
Kaggle<https://www.kaggle.com/code/msripooja/breast-cancer-prediction-using-svm> Accessed 25 December 2024.
- raj kg, pavan (2024) *Accuracy : 98.25 % model using Logistic Regression with Hyperparameter Tuning*.
https://github.com/pavanrajkg04/Machine_Learning_projects Accessed 2 November 2024.
- Ramirez, A. (2021) *3 Most Popular Data Science Metho*. <https://medium.com/@aj.ramirez23/3-most-popular-data-science-metho-e61f6600b83f> Accessed.
- S, P. (2021) *Support Vector Machine | Beginners Guide to Support Vector Machine*.
<https://www.analyticsvidhya.com/blog/2021/06/support-vector-machine-better-understanding/>
Accessed.
- saikat (2021) *5 Techniques to Handle Imbalanced Data For a Classification Problem*.
<https://www.analyticsvidhya.com/blog/2021/06/5-techniques-to-handle-imbalanced-data-for-a-classification-problem/#h-2-resampling-oversampling-and-undersampling> Accessed 25 December 2024.
- scikit-learn (2010) *sklearn.neural_network.MLPClassifier — scikit-learn 0.20.3 documentation*.
https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
Accessed.
- Singh, A.P. (2020) *Steps You Should Follow TO Successfully Train MLP*. <https://medium.com/analytics-vidhya/steps-you-should-follow-to-successfully-train-mlp-40a98c3b5bb3> Accessed.
- Singh, S. (2023) *Top 10 Interview Questions on Evaluation Metrics in Machine Learning*. Towards AI <https://pub.towardsai.net/top-10-interview-questions-on-evaluation-metrics-in-machine-learning-407c547e7b46> Accessed 25 December 2024.

Szymon Plotka (2018) *Breast cancer classification using scikit-learn and Keras - Ermlab Software*.

<https://ermlab.com/en/blog/data-science/breast-cancer-classification-using-scikit-learn-and-keras/>

Accessed.

World Health Organization (2024) *Breast Cancer*. [https://www.who.int/news-room/fact-](https://www.who.int/news-room/fact-sheets/detail/breast-cancer)

[sheets/detail/breast-cancer](https://www.who.int/news-room/fact-sheets/detail/breast-cancer) Accessed.